

Human review packet: GS62CollegeAdmissions

Generated from byte-pinned source excerpts, the expanded PaperInterface specifications, and hash-bound raw-source-to-Spec screening records.

Paper: GS62CollegeAdmissions

Paper id: GS62CollegeAdmissions

Source version: American Mathematical Monthly 69(1), January 1962, printed pages 9-15

Source record: audit/paper_statement_map.json

Rows: 5

Generated: 2026-08-18

Source URL: [official source](#)

How to use this packet

For each source claim, compare the exact source excerpts with the expanded PaperInterface specification. Mark up this PDF or fill the blank reviewer box in the TeX source; return the annotated file for reconciliation into the paper-local human-review log.

Open the interactive dashboard

The interactive dashboard is an optional alternative to using this PDF; it presents the same review surface rather than an additional review requirement. After forking and cloning (or pulling) EconCSLib, run the following from the repository root. The cache command is needed only the first time in a new clone.

```
cd <your-EconCSLib-checkout>
lake exe cache get
python3 scripts/review_dashboard.py --paper GS62CollegeAdmissions --serve
```

Open <http://127.0.0.1:8765/> in a browser and keep that terminal running while reviewing. The dashboard records saved annotations in this paper's local review trace.

Contents

- Semantic library prerequisites (6; not paper claims)
 - [EconCSLib.FiniteChoice.linearTopQChoice](#)
 - [Assignment M W](#)
 - [ManyToOne.valApplicant](#)
 - [ManyToOneAssignment Applicants Colleges](#)
 - [valM](#)
 - [valW](#)
- Paper-local semantic prerequisites (22; not paper claims)
 - [GS62CollegeAdmissions.ExactCollegeBatchedProcedure.SourceWaitingListState](#)
 - [GS62CollegeAdmissions.ExactCollegeBatchedProcedure.collegeOrder](#)
 - [GS62CollegeAdmissions.ExactCollegeBatchedProcedure.collegeTopQ](#)
 - [GS62CollegeAdmissions.ExactCollegeBatchedProcedure.waitingList](#)
 - [GS62CollegeAdmissions.ExactCollegeBatchedProcedure.assignedCollege](#)
 - [GS62CollegeAdmissions.ExactCollegeBatchedProcedure.sourceEligible](#)
 - [GS62CollegeAdmissions.ExactCollegeBatchedProcedure.untriedEligibleColleges](#)
 - [GS62CollegeAdmissions.ExactCollegeBatchedProcedure.SourceActive](#)
 - [GS62CollegeAdmissions.ExactCollegeBatchedProcedure.nextCollege](#)
 - [GS62CollegeAdmissions.ExactCollegeBatchedProcedure.newApplications](#)
 - [GS62CollegeAdmissions.ExactCollegeBatchedProcedure.sourceStep](#)
 - [GS62CollegeAdmissions.ExactCollegeBatchedProcedure.SourceStepRelation](#)
 - [GS62CollegeAdmissions.ExactCollegeBatchedProcedure.initialSourceState](#)
 - [GS62CollegeAdmissions.ExactCollegeBatchedProcedure.SourceReachable](#)
 - [GS62CollegeAdmissions.ExactCollegeBatchedProcedure.sourceAssignmentView](#)
 - [GS62CollegeAdmissions.gs62ReplacementPair](#)
 - [GS62CollegeAdmissions.gs62LiteralStableCollegeAssignment](#)
 - [GS62CollegeAdmissions.gs62LiteralApplicantOptimalCollegeAssignment](#)
 - [GS62CollegeAdmissions.PaperInterface.literalApplicantOptimalCollegeAssignment](#)
 - [GS62CollegeAdmissions.PaperInterface.replacementPairCollegeInstability](#)
 - [GS62CollegeAdmissions.PaperInterface.unstableCollegeAssignment](#)

- GS62CollegeAdmissions.PaperInterface.unstableMarriage
- Main-text source claims (5)
 - unstableCollegeAssignment_iff_source_definitionSpec
 - literalApplicantOptimalCollegeAssignment_iff_source_definitionSpec
 - unstableMarriage_iff_source_definitionSpec
 - theorem1_stable_marriage_existsSpec
 - theorem2_applicant_optimalitySpec

Material library prerequisites

EconCSLib.FiniteChoice.linearTopQChoice

Source locator: source.txt:232-249; source.txt:253-255 (printed pages 13-14, Sections 4-5 and Theorem 2)

Verbatim paper-source connection

4. Stable assignments and the admissions problem. The extension of our "deferred-acceptance" procedure to the problem of college admissions is straightforward. For convenience we will assume that if a college is not willing to accept a student under any circumstances, as described in Section 2, then that student will not even be permitted to apply to the college. With this understanding the procedure follows: First, all students apply to the college of their first choice. A college with a quota of q then places on its waiting list the q applicants who rank highest, or all applicants if there are fewer than q , and rejects the rest. Rejected applicants then apply to their second choice and again each college selects the top q from among the new applicants and those on its waiting list, puts these on its new waiting list, and rejects the rest. The procedure terminates when every applicant is either on a waiting list or has been rejected by every college to which he is willing and permitted to apply. At this point each college

[form-feed in source extraction]
14 COLLEGE ADMISSIONS AND STABILITY OF MARRIAGE [January

admits everyone on its waiting list and the stable assignment has been achieved. The proof that the assignment is stable is entirely analogous to the proof given for the marriage problem and is left to the reader.

[Next verbatim source excerpt]

THEOREM 2. Every applicant is at least as well off under the assignment given by the deferred acceptance procedure as he would be under any other stable assignment.

[Next verbatim source excerpt]

DEFINITION. An assignment of applicants to colleges will be called unstable if there are two applicants a and b who are assigned to colleges A and B , respectively, although A to B and A prefers: $to a$.
, 3 prefers

[Next verbatim source excerpt]

4. Stable assignments and the admissions problem. The extension of our "deferred-acceptance" procedure to the problem of college admissions is straightforward. For convenience we will assume that if a college is not willing to accept a student under any circumstances, as described in Section 2, then that student will not even be permitted to apply to the college. With this understanding the procedure follows: First, all students apply to the college of their first choice. A college with a quota of q then places on its waiting list the q applicants who rank highest, or all applicants if there are fewer than q , and rejects the rest. Rejected applicants then apply to their second choice and again each college selects the top q from among the new applicants and those on its waiting list, puts these on its new waiting list, and rejects the rest. The procedure terminates when every applicant is either on a waiting list or has been rejected by every college to which he is willing and permitted to apply. At this point each college

[form-feed in source extraction]
14 COLLEGE ADMISSIONS AND STABILITY OF MARRIAGE [January

admits everyone on its waiting list and the stable assignment has been achieved. The proof that the assignment is stable is entirely analogous to the proof given for the marriage problem and is left to the reader.

[Next verbatim source excerpt]

5. Optimality. We now show that the "deferred acceptance" procedure just described yields not only a stable but an optimal assignment of applicants. That is,

Lean-expanded library semantic target

```
{α : Type u_1} →  
[inst : DecidableEq α] →  
[inst_1 : LinearOrder α] →  
  (q : ℕ) →  
    (a : Finset α) →  
      if h : q ≤ a.card then Finset.image (fun i => ↑((a.orderIsoOfFin ↦) (↑i, ...))) Finset.univ else a
```

Exact Lean library declaration

```
noncomputable def linearTopQChoice (q : ℕ) : ChoiceRule α :=  
  fun X =>  
    if h : q ≤ X.card then  
      Finset.univ.image  
        (fun i : Fin q =>  
          ((X.orderIsoOfFin (by rfl)  
            (i.1, lt_of_lt_of_le i.2 h) : X) : α))  
    else  
      X
```

Recorded source-to-library screening **Verdict:** matches

Reason: The source says each college retains the q applicants it ranks highest, or all applicants if fewer apply. The Lean library function is exactly that top- q selection from the candidate pool under the college order.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Assignment M W

Source locator: source.txt:133-139 (printed page 11, Section 3)

Verbatim paper-source connection

A certain community consists of n men and n women. Each person ranks those of the opposite sex in accordance with his or her preferences for a marriage partner. We seek a satisfactory way of marrying off all members of the community. Imitating our earlier definition, we call a set of marriages unstable (and here the suitability of the term is quite clear) if under it there are a man and a woman who are not married to each other but prefer each other to their actual mates.

Lean declaration type (metadata)

Type $u_1 \rightarrow \text{Type } u_2 \rightarrow \text{Type } (\max u_1 u_2)$

Exact Lean library declaration

```
-- A matching between Men and Women. -/
structure Assignment (M W : Type*) where
  m_match : M → Option W
  w_match : W → Option M
  consistent_m : ∀ m w, m_match m = some w ↔ w_match w = some m
```

Recorded source-to-library screening Verdict: matches

Reason: The source's set of marriages pairs each participant with an actual mate. The Lean structure records both partner maps and their mutual consistency; the PaperInterface predicate separately states the source's complete-marriage domain.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

ManyToOne.valApplicant

Source locator: source.txt:232-249; source.txt:253-255 (printed pages 13-14, Sections 4-5 and Theorem 2)

Verbatim paper-source connection

4. Stable assignments and the admissions problem. The extension of our "deferred-acceptance" procedure to the problem of college admissions is straightforward. For convenience we will assume that if a college is not willing to accept a student under any circumstances, as described in Section 2, then that student will not even be permitted to apply to the college. With this understanding the procedure follows: First, all students apply to the college of their first choice. A college with a quota of q then places on its waiting list the q applicants who rank highest, or all applicants if there are fewer than q , and rejects the rest. Rejected applicants then apply to their second choice and again each college selects the top q from among the new applicants and those on its waiting list, puts these on its new waiting list, and rejects the rest. The procedure terminates when every applicant is either on a waiting list or has been rejected by every college to which he is willing and permitted to apply. At this point each college

[form-feed in source extraction]
14 COLLEGE ADMISSIONS AND STABILITY OF MARRIAGE [January

admits everyone on its waiting list and the stable assignment has been achieved. The proof that the assignment is stable is entirely analogous to the proof given for the marriage problem and is left to the reader.

[Next verbatim source excerpt]

THEOREM 2. Every applicant is at least as well off under the assignment given by the deferred acceptance procedure as he would be under any other stable assignment.

[Next verbatim source excerpt]

DEFINITION. An assignment of applicants to colleges will be called unstable if there are two applicants a and b who are assigned to colleges A and B , respectively, although A to b and A prefers: $to a$.
 a prefers b .

[Next verbatim source excerpt]

4. Stable assignments and the admissions problem. The extension of our "deferred-acceptance" procedure to the problem of college admissions is straightforward. For convenience we will assume that if a college is not willing to accept a student under any circumstances, as described in Section 2, then that student will not even be permitted to apply to the college. With this understanding the procedure follows: First, all students apply to the college of their first choice. A college with a quota of q then places on its waiting list the q applicants who rank highest, or all applicants if there are fewer than q , and rejects the rest. Rejected applicants then apply to their second choice and again each college selects the top q from among the new applicants and those on its waiting list, puts these on its new waiting list, and rejects the rest. The procedure terminates when every applicant is either on a waiting list or has been rejected by every college to which he is willing and permitted to apply. At this point each college

[form-feed in source extraction]
14 COLLEGE ADMISSIONS AND STABILITY OF MARRIAGE [January

admits everyone on its waiting list and the stable assignment has been achieved. The proof that the assignment is stable is entirely analogous to the proof given for the marriage problem and is left to the reader.

[Next verbatim source excerpt]

5. Optimality. We now show that the "deferred acceptance" procedure just described yields not only a stable but an optimal assignment of applicants. That is,

Lean-expanded library semantic target

```
{Applicants : Type u_1} →  
{Colleges : Type u_2} →  
(val_applicant : Applicants → Colleges → ℝ) →  
(a : Applicants) →  
(c : Option Colleges) →  
  match c with  
  | none => 0  
  | some c' => val_applicant a c'
```

Exact Lean library declaration

```
def valApplicant {Applicants Colleges : Type*}  
  (val_applicant : Applicants → Colleges → ℝ)  
  (a : Applicants) (c : Option Colleges) : ℝ :=  
  match c with  
  | none => 0  
  | some c' => val_applicant a c'
```

Recorded source-to-library screening **Verdict:** matches

Reason: Sections 4–5 make a terminal applicant either waitlisted or rejected by every willing and permitted college. The procedure’s applicant comparison uses an explicit zero outside option for that unmatched terminal case; when matched, the definition returns exactly the applicant’s value for the assigned college.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

ManyToOneAssignment Applicants Colleges

Source locator: source.txt:93-98 (printed page 10, first displayed Definition)

Verbatim paper-source connection

DEFINITION. An assignment of applicants to colleges will be called unstable if there are two applicants a and b who are assigned to colleges A and B , respectively, although a to B and A prefers a to b .

Lean declaration type (metadata)

Type $u_1 \rightarrow \text{Type } u_2 \rightarrow \text{Type } (\max u_1 u_2)$

Exact Lean library declaration

```
-- A many-to-one assignment between applicants and colleges. -/
structure ManyToOneAssignment (Applicants Colleges : Type*) where
  app_match : Applicants → Option Colleges
  college_roster : Colleges → Finset Applicants
  consistent : ∀ a c, app_match a = some c ↔ a ∈ college_roster c
```

Recorded source-to-library screening Verdict: matches

Reason: The source describes applicants assigned to colleges and later permits an applicant to be rejected by every college willing to consider them. The Lean structure records the applicant-to-college relation, college rosters, and their consistency, including that permitted unmatched outcome.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

valM

Source locator: source.txt:133-139 (printed page 11, Section 3)

Verbatim paper-source connection

A certain community consists of n men and n women. Each person ranks those of the opposite sex in accordance with his or her preferences for a marriage partner. We seek a satisfactory way of marrying off all members of the community. Imitating our earlier definition, we call a set of marriages unstable (and here the suitability of the term is quite clear) if under it there are a man and a woman who are not married to each other but prefer each other to their actual mates.

Lean-expanded library semantic target

```
{M : Type u_1} →
{W : Type u_2} →
(val : M → W → ℝ) →
(m : M) →
(w : Option W) →
match w with
| none => 0
| some w' => val m w'
```

Exact Lean library declaration

```
-- The value of a match for a man. 0 if unmatched. -/
def valM {M W : Type*} (val : M → W → ℝ) (m : M) (w : Option W) : ℝ :=
  match w with
  | none => 0
  | some w' => val m w'
```

Recorded source-to-library screening Verdict: matches

Reason: On the explicitly complete marriage domain of the reviewed source claim, the only relevant branch returns the source man's value for his actual mate; the none-to-zero branch does not represent a source actual mate.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

valW

Source locator: source.txt:133-139 (printed page 11, Section 3)

Verbatim paper-source connection

A certain community consists of n men and n women. Each person ranks those of the opposite sex in accordance with his or her preferences for a marriage partner. We seek a satisfactory way of marrying off all members of the community. Imitating our earlier definition, we call a set of marriages unstable (and here the suitability of the term is quite clear) if under it there are a man and a woman who are not married to each other but prefer each other to their actual mates.

Lean-expanded library semantic target

```
{M : Type u_1} →
{W : Type u_2} →
(val : W → M → ℝ) →
(w : W) →
(m : Option M) →
  match m with
  | none => 0
  | some m' => val w m'
```

Exact Lean library declaration

```
-- The value of a match for a woman. 0 if unmatched. -/
def valW {M W : Type*} (val : W → M → ℝ) (w : W) (m : Option M) : ℝ :=
  match m with
  | none => 0
  | some m' => val w m'
```

Recorded source-to-library screening Verdict: matches

Reason: On the explicitly complete marriage domain of the reviewed source claim, the only relevant branch returns the source woman's value for her actual mate; the none-to-zero branch does not represent a source actual mate.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Paper-specific semantic prerequisites

These paper-local state, policy, or transition declarations remain named in the Lean-expanded claim because they are independent semantic objects. Each is shown with its own verbatim source connection and reviewer annotation before the claim that uses it. They are prerequisites, not extra paper-claim rows.

GS62CollegeAdmissions.ExactCollegeBatchedProcedure.SourceWaitingListState

Paper-local declaration:

GS62CollegeAdmissions.ExactCollegeBatchedProcedure.SourceWaitingListState

Source locator: source.txt:232-249

Verbatim paper-source connection

4. Stable assignments and the admissions problem. The extension of our "deferred-acceptance" procedure to the problem of college admissions is straightforward. For convenience we will assume that if a college is not willing to accept a student under any circumstances, as described in Section 2, then that student will not even be permitted to apply to the college. With this understanding the procedure follows: First, all students apply to the college of their first choice. A college with a quota of q then places on its waiting list the q applicants who rank highest, or all applicants if there are fewer than q , and rejects the rest. Rejected applicants then apply to their second choice and again each college selects the top q from among the new applicants and those on its waiting list, puts these on its new waiting list, and rejects the rest. The procedure terminates when every applicant is either on a waiting list or has been rejected by every college to which he is willing and permitted to apply. At this point each college

[form-feed in source extraction]
14 COLLEGE ADMISSIONS AND STABILITY OF MARRIAGE [January

admits everyone on its waiting list and the stable assignment has been achieved. The proof that the assignment is stable is entirely analogous to the proof given for the marriage problem and is left to the reader.

Lean-elaborated paper signature

(Applicants : Type u_3) → Type u_4 → [DecidableEq Applicants] → Type (max u_3 u_4)

Recorded source-to-declaration screening Verdict: matches

Reason: Sections 4–5 describe applicants applying in rounds, colleges retaining a quota-sized waiting list, and the terminal lists determining assignments. The Lean state records exactly the per-college waiting lists needed for that source procedure.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GS62CollegeAdmissions.ExactCollegeBatchedProcedure.collegeOrder

Paper-local declaration:

GS62CollegeAdmissions.ExactCollegeBatchedProcedure.collegeOrder

Source locator: source.txt:232-249; source.txt:253-255 (printed pages 13-14, Sections 4-5 and Theorem 2)

Verbatim paper-source connection

4. Stable assignments and the admissions problem. The extension of our "deferred-acceptance" procedure to the problem of college admissions is straightforward. For convenience we will assume that if a college is not willing to accept a student under any circumstances, as described in Section 2, then that student will not even be permitted to apply to the college. With this understanding the procedure follows: First, all students apply to the college of their first choice. A college with a quota of q then places on its waiting list the q applicants who rank highest, or all applicants if there are fewer than q , and rejects the rest. Rejected applicants then apply to their second choice and again each college selects the top q from among the new applicants and those on its waiting list, puts these on its new waiting list, and rejects the rest. The procedure terminates when every applicant is either on a waiting list or has been rejected by every college to which he is willing and permitted to apply. At this point each college

[form-feed in source extraction]

14 COLLEGE ADMISSIONS AND STABILITY OF MARRIAGE [January

admits everyone on its waiting list and the stable assignment has been achieved. The proof that the assignment is stable is entirely analogous to the proof given for the marriage problem and is left to the reader.

[Next verbatim source excerpt]

THEOREM 2. Every applicant is at least as well off under the assignment given by the deferred acceptance procedure as he would be under any other stable assignment.

[Next verbatim source excerpt]

DEFINITION. An assignment of applicants to colleges will be called unstable if there are two applicants a and b who are assigned to colleges A and B , respectively, although A to b and A prefers: $to a$.
 b prefers

[Next verbatim source excerpt]

4. Stable assignments and the admissions problem. The extension of our "deferred-acceptance" procedure to the problem of college admissions is straightforward. For convenience we will assume that if a college is not willing to accept a student under any circumstances, as described in Section 2, then that student will not even be permitted to apply to the college. With this understanding the procedure follows: First, all students apply to the college of their first choice. A college with a quota of q then places on its waiting list the q applicants who rank highest, or all applicants if there are fewer than q , and rejects the rest. Rejected applicants then apply to their second choice and again each college selects the top q from among the new applicants and those on its waiting list, puts these on its new waiting list, and rejects the rest. The procedure terminates when every applicant is either on a waiting list or has been rejected by every college to which he is willing and permitted to apply. At this point each college

[form-feed in source extraction]

14 COLLEGE ADMISSIONS AND STABILITY OF MARRIAGE [January

admits everyone on its waiting list and the stable assignment has been achieved. The proof that the assignment is stable is entirely analogous to the proof given for the marriage problem and is left to the reader.

[Next verbatim source excerpt]

5. Optimality. We now show that the "deferred acceptance" procedure just described yields not only a stable but an optimal assignment of applicants. That is,

Lean-expanded paper semantic target

```
{Applicants : Type u_1} →
{Colleges : Type u_2} →
(val_college : Colleges → Applicants → ℝ) →
(hcollegeStrict : ∀ (c : Colleges) (a a' : Applicants), val_college c a = val_college c a' → a = a') →
(c : Colleges) → LinearOrder.lift' (fun a => OrderDual.toDual (val_college c a)) ...
```

Recorded source-to-declaration screening Verdict: matches

Reason: Each college retains the applicants it ranks highest. The Lean order is the strict college ranking used to make that top- q choice.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GS62CollegeAdmissions.ExactCollegeBatchedProcedure.collegeTopQ

Paper-local declaration:

GS62CollegeAdmissions.ExactCollegeBatchedProcedure.collegeTopQ

Source locator: source.txt:232-249; source.txt:253-255 (printed pages 13-14, Sections 4-5 and Theorem 2)

Verbatim paper-source connection

4. Stable assignments and the admissions problem. The extension of our "deferred-acceptance" procedure to the problem of college admissions is straightforward. For convenience we will assume that if a college is not willing to accept a student under any circumstances, as described in Section 2, then that student will not even be permitted to apply to the college. With this understanding the procedure follows: First, all students apply to the college of their first choice. A college with a quota of q then places on its waiting list the q applicants who rank highest, or all applicants if there are fewer than q , and rejects the rest. Rejected applicants then apply to their second choice and again each college selects the top q from among the new applicants and those on its waiting list, puts these on its new waiting list, and rejects the rest. The procedure terminates when every applicant is either on a waiting list or has been rejected by every college to which he is willing and permitted to apply. At this point each college

[form-feed in source extraction]

14 COLLEGE ADMISSIONS AND STABILITY OF MARRIAGE [January

admits everyone on its waiting list and the stable assignment has been achieved. The proof that the assignment is stable is entirely analogous to the proof given for the marriage problem and is left to the reader.

[Next verbatim source excerpt]

THEOREM 2. Every applicant is at least as well off under the assignment given by the deferred acceptance procedure as he would be under any other stable assignment.

[Next verbatim source excerpt]

DEFINITION. An assignment of applicants to colleges will be called unstable if there are two applicants a and b who are assigned to colleges A and B , respectively, although A to b and A prefers: $to a$.
 b prefers

[Next verbatim source excerpt]

4. Stable assignments and the admissions problem. The extension of our "deferred-acceptance" procedure to the problem of college admissions is straightforward. For convenience we will assume that if a college is not willing to accept a student under any circumstances, as described in Section 2, then that student will not even be permitted to apply to the college. With this understanding the procedure follows: First, all students apply to the college of their first choice. A college with a quota of q then places on its waiting list the q applicants who rank highest, or all applicants if there are fewer than q , and rejects the rest. Rejected applicants then apply to their second choice and again each college selects the top q from among the new applicants and those on its waiting list, puts these on its new waiting list, and rejects the rest. The procedure terminates when every applicant is either on a waiting list or has been rejected by every college to which he is willing and permitted to apply. At this point each college

[form-feed in source extraction]

14 COLLEGE ADMISSIONS AND STABILITY OF MARRIAGE [January

admits everyone on its waiting list and the stable assignment has been achieved. The proof that the assignment is stable is entirely analogous to the proof given for the marriage problem and is left to the reader.

[Next verbatim source excerpt]

5. Optimality. We now show that the "deferred acceptance" procedure just described yields not only a stable but an optimal assignment of applicants. That is,

Lean-expanded paper semantic target

```
{Applicants : Type u_1} →
{Colleges : Type u_2} →
[inst : DecidableEq Applicants] →
(quota : Colleges → ℕ) →
(val_college : Colleges → Applicants → ℝ) →
(hCollegeStrict : ∀ (c : Colleges) (a a' : Applicants), val_college c a = val_college c a' → a = a') →
(c : Colleges) → (pool : Finset Applicants) → EconCSLib.FiniteChoice.linearTopQChoice (quota c) pool
```

Recorded source-to-declaration screening Verdict: matches

Reason: The source says a college places its q highest-ranked applicants, or all applicants if fewer, on its waiting list. The Lean function is that literal top- q selection.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GS62CollegeAdmissions.ExactCollegeBatchedProcedure.waitingList

Paper-local declaration:

GS62CollegeAdmissions.ExactCollegeBatchedProcedure.waitingList

Source locator: source.txt:232-249; source.txt:253-255 (printed pages 13-14, Sections 4-5 and Theorem 2)

Verbatim paper-source connection

4. Stable assignments and the admissions problem. The extension of our "deferred-acceptance" procedure to the problem of college admissions is straightforward. For convenience we will assume that if a college is not willing to accept a student under any circumstances, as described in Section 2, then that student will not even be permitted to apply to the college. With this understanding the procedure follows: First, all students apply to the college of their first choice. A college with a quota of q then places on its waiting list the q applicants who rank highest, or all applicants if there are fewer than q , and rejects the rest. Rejected applicants then apply to their second choice and again each college selects the top q from among the new applicants and those on its waiting list, puts these on its new waiting list, and rejects the rest. The procedure terminates when every applicant is either on a waiting list or has been rejected by every college to which he is willing and permitted to apply. At this point each college

[form-feed in source extraction]

14 COLLEGE ADMISSIONS AND STABILITY OF MARRIAGE [January

admits everyone on its waiting list and the stable assignment has been achieved. The proof that the assignment is stable is entirely analogous to the proof given for the marriage problem and is left to the reader.

[Next verbatim source excerpt]

THEOREM 2. Every applicant is at least as well off under the assignment given by the deferred acceptance procedure as he would be under any other stable assignment.

[Next verbatim source excerpt]

DEFINITION. An assignment of applicants to colleges will be called unstable if there are two applicants a and b who are assigned to colleges A and B , respectively, although A to b and A prefers: $to a$.
 $,3$ prefers

[Next verbatim source excerpt]

4. Stable assignments and the admissions problem. The extension of our "deferred-acceptance" procedure to the problem of college admissions is straightforward. For convenience we will assume that if a college is not willing to accept a student under any circumstances, as described in Section 2, then that student will not even be permitted to apply to the college. With this understanding the procedure follows: First, all students apply to the college of their first choice. A college with a quota of q then places on its waiting list the q applicants who rank highest, or all applicants if there are fewer than q , and rejects the rest. Rejected applicants then apply to their second choice and again each college selects the top q from among the new applicants and those on its waiting list, puts these on its new waiting list, and rejects the rest. The procedure terminates when every applicant is either on a waiting list or has been rejected by every college to which he is willing and permitted to apply. At this point each college

[form-feed in source extraction]

14 COLLEGE ADMISSIONS AND STABILITY OF MARRIAGE [January

admits everyone on its waiting list and the stable assignment has been achieved. The proof that the assignment is stable is entirely analogous to the proof given for the marriage problem and is left to the reader.

[Next verbatim source excerpt]

5. Optimality. We now show that the "deferred acceptance" procedure just described yields not only a stable but an optimal assignment of applicants. That is,

Lean-expanded paper semantic target

```
{Applicants : Type u_1} →
{Colleges : Type u_2} →
[inst : DecidableEq Applicants] →
(quota : Colleges → ℕ) →
(val_college : Colleges → Applicants → ℝ) →
(hCollegeStrict : ∀ (c : Colleges) (a a' : Applicants), val_college c a = val_college c a' → a = a') →
(s : GS62CollegeAdmissions.ExactCollegeBatchedProcedure.SourceWaitingListState Applicants Colleges) →
(c : Colleges) →
GS62CollegeAdmissions.ExactCollegeBatchedProcedure.collegeTopQ quota val_college hCollegeStrict c
(s.applications c)
```

Recorded source-to-declaration screening Verdict: matches

Reason: The source repeatedly forms a college waiting list by retaining its highest-ranked applicants among the old list and new applications. The Lean definition is that list at a procedure state.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GS62CollegeAdmissions.ExactCollegeBatchedProcedure.assignedCollege

Paper-local declaration:

GS62CollegeAdmissions.ExactCollegeBatchedProcedure.assignedCollege

Source locator: source.txt:232-249; source.txt:253-255 (printed pages 13-14, Sections 4-5 and Theorem 2)

Verbatim paper-source connection

4. Stable assignments and the admissions problem. The extension of our "deferred-acceptance" procedure to the problem of college admissions is straightforward. For convenience we will assume that if a college is not willing to accept a student under any circumstances, as described in Section 2, then that student will not even be permitted to apply to the college. With this understanding the procedure follows: First, all students apply to the college of their first choice. A college with a quota of q then places on its waiting list the q applicants who rank highest, or all applicants if there are fewer than q , and rejects the rest. Rejected applicants then apply to their second choice and again each college selects the top q from among the new applicants and those on its waiting list, puts these on its new waiting list, and rejects the rest. The procedure terminates when every applicant is either on a waiting list or has been rejected by every college to which he is willing and permitted to apply. At this point each college

[form-feed in source extraction]

14 COLLEGE ADMISSIONS AND STABILITY OF MARRIAGE [January

admits everyone on its waiting list and the stable assignment has been achieved. The proof that the assignment is stable is entirely analogous to the proof given for the marriage problem and is left to the reader.

[Next verbatim source excerpt]

THEOREM 2. Every applicant is at least as well off under the assignment given by the deferred acceptance procedure as he would be under any other stable assignment.

[Next verbatim source excerpt]

DEFINITION. An assignment of applicants to colleges will be called unstable if there are two applicants a and b who are assigned to colleges A and B , respectively, although A to b and A prefers: toa .
 a prefers b .

[Next verbatim source excerpt]

4. Stable assignments and the admissions problem. The extension of our "deferred-acceptance" procedure to the problem of college admissions is straightforward. For convenience we will assume that if a college is not willing to accept a student under any circumstances, as described in Section 2, then that student will not even be permitted to apply to the college. With this understanding the procedure follows: First, all students apply to the college of their first choice. A college with a quota of q then places on its waiting list the q applicants who rank highest, or all applicants if there are fewer than q , and rejects the rest. Rejected applicants then apply to their second choice and again each college selects the top q from among the new applicants and those on its waiting list, puts these on its new waiting list, and rejects the rest. The procedure terminates when every applicant is either on a waiting list or has been rejected by every college to which he is willing and permitted to apply. At this point each college

[form-feed in source extraction]

14 COLLEGE ADMISSIONS AND STABILITY OF MARRIAGE [January

admits everyone on its waiting list and the stable assignment has been achieved. The proof that the assignment is stable is entirely analogous to the proof given for the marriage problem and is left to the reader.

[Next verbatim source excerpt]

5. Optimality. We now show that the "deferred acceptance" procedure just described yields not only a stable but an optimal assignment of applicants. That is,

Lean-expanded paper semantic target

```
{Applicants : Type u_1} →
{Colleges : Type u_2} →
[inst : Fintype Colleges] →
[inst_1 : DecidableEq Applicants] →
(quota : Colleges → ℕ) →
(val_college : Colleges → Applicants → ℝ) →
(hcollegeStrict : ∀ (c : Colleges) (a a' : Applicants), val_college c a = val_college c a' → a = a') →
(s : GS62CollegeAdmissions.ExactCollegeBatchedProcedure.SourceWaitingListState Applicants Colleges) →
(a : Applicants) →
if h :
  if h :
    ∃ c,
      a ∈
        GS62CollegeAdmissions.ExactCollegeBatchedProcedure.waitingList quota val_college
        hcollegeStrict s c then
      some (Classical.choose h)
    else none
```

Recorded source-to-declaration screening Verdict: matches

Reason: At termination the source admits each applicant on a college waiting list and leaves every other applicant unassigned. The Lean function reads that assignment from the lists.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GS62CollegeAdmissions.ExactCollegeBatchedProcedure.sourceEligible

Paper-local declaration:

GS62CollegeAdmissions.ExactCollegeBatchedProcedure.sourceEligible

Source locator: source.txt:232-249; source.txt:253-255 (printed pages 13-14, Sections 4-5 and Theorem 2)

Verbatim paper-source connection

4. Stable assignments and the admissions problem. The extension of our "deferred-acceptance" procedure to the problem of college admissions is straightforward. For convenience we will assume that if a college is not willing to accept a student under any circumstances, as described in Section 2, then that student will not even be permitted to apply to the college. With this understanding the procedure follows: First, all students apply to the college of their first choice. A college with a quota of q then places on its waiting list the q applicants who rank highest, or all applicants if there are fewer than q , and rejects the rest. Rejected applicants then apply to their second choice and again each college selects the top q from among the new applicants and those on its waiting list, puts these on its new waiting list, and rejects the rest. The procedure terminates when every applicant is either on a waiting list or has been rejected by every college to which he is willing and permitted to apply. At this point each college

[form-feed in source extraction]

14 COLLEGE ADMISSIONS AND STABILITY OF MARRIAGE [January

admits everyone on its waiting list and the stable assignment has been achieved. The proof that the assignment is stable is entirely analogous to the proof given for the marriage problem and is left to the reader.

[Next verbatim source excerpt]

THEOREM 2. Every applicant is at least as well off under the assignment given by the deferred acceptance procedure as he would be under any other stable assignment.

[Next verbatim source excerpt]

DEFINITION. An assignment of applicants to colleges will be called unstable if there are two applicants a and b who are assigned to colleges A and B , respectively, although A to b and A prefers: $to a$.
 b prefers

[Next verbatim source excerpt]

4. Stable assignments and the admissions problem. The extension of our "deferred-acceptance" procedure to the problem of college admissions is straightforward. For convenience we will assume that if a college is not willing to accept a student under any circumstances, as described in Section 2, then that student will not even be permitted to apply to the college. With this understanding the procedure follows: First, all students apply to the college of their first choice. A college with a quota of q then places on its waiting list the q applicants who rank highest, or all applicants if there are fewer than q , and rejects the rest. Rejected applicants then apply to their second choice and again each college selects the top q from among the new applicants and those on its waiting list, puts these on its new waiting list, and rejects the rest. The procedure terminates when every applicant is either on a waiting list or has been rejected by every college to which he is willing and permitted to apply. At this point each college

[form-feed in source extraction]

14 COLLEGE ADMISSIONS AND STABILITY OF MARRIAGE [January

admits everyone on its waiting list and the stable assignment has been achieved. The proof that the assignment is stable is entirely analogous to the proof given for the marriage problem and is left to the reader.

[Next verbatim source excerpt]

5. Optimality. We now show that the "deferred acceptance" procedure just described yields not only a stable but an optimal assignment of applicants. That is,

Lean-expanded paper semantic target

$\forall \{ \text{Applicants} : \text{Type } u_1 \} \{ \text{Colleges} : \text{Type } u_2 \} (\text{val_applicant} : \text{Applicants} \rightarrow \text{Colleges} \rightarrow \mathbb{R})$
 $(\text{val_college} : \text{Colleges} \rightarrow \text{Applicants} \rightarrow \mathbb{R}) (\text{a} : \text{Applicants}) (\text{c} : \text{Colleges}), 0 < \text{val_applicant } a \text{ c} \wedge 0 < \text{val_college } c \text{ a}$

Recorded source-to-declaration screening Verdict: matches

Reason: The source excludes colleges unwilling to accept a student and applicants not willing or permitted to apply. The Lean predicate records precisely the mutual eligibility condition.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GS62CollegeAdmissions.ExactCollegeBatchedProcedure.untriedEligibleColleges

Paper-local declaration:

GS62CollegeAdmissions.ExactCollegeBatchedProcedure.untriedEligibleColleges

Source locator: source.txt:232-249; source.txt:253-255 (printed pages 13-14, Sections 4-5 and Theorem 2)

Verbatim paper-source connection

4. Stable assignments and the admissions problem. The extension of our "deferred-acceptance" procedure to the problem of college admissions is straightforward. For convenience we will assume that if a college is not willing to accept a student under any circumstances, as described in Section 2, then that student will not even be permitted to apply to the college. With this understanding the procedure follows: First, all students apply to the college of their first choice. A college with a quota of q then places on its waiting list the q applicants who rank highest, or all applicants if there are fewer than q , and rejects the rest. Rejected applicants then apply to their second choice and again each college selects the top q from among the new applicants and those on its waiting list, puts these on its new waiting list, and rejects the rest. The procedure terminates when every applicant is either on a waiting list or has been rejected by every college to which he is willing and permitted to apply. At this point each college

[form-feed in source extraction]

14 COLLEGE ADMISSIONS AND STABILITY OF MARRIAGE [January

admits everyone on its waiting list and the stable assignment has been achieved. The proof that the assignment is stable is entirely analogous to the proof given for the marriage problem and is left to the reader.

[Next verbatim source excerpt]

THEOREM 2. Every applicant is at least as well off under the assignment given by the deferred acceptance procedure as he would be under any other stable assignment.

[Next verbatim source excerpt]

DEFINITION. An assignment of applicants to colleges will be called unstable if there are two applicants a and b who are assigned to colleges A and B , respectively, although A to b and A prefers: toa , b prefers

[Next verbatim source excerpt]

4. Stable assignments and the admissions problem. The extension of our "deferred-acceptance" procedure to the problem of college admissions is straightforward. For convenience we will assume that if a college is not willing to accept a student under any circumstances, as described in Section 2, then that student will not even be permitted to apply to the college. With this understanding the procedure follows: First, all students apply to the college of their first choice. A college with a quota of q then places on its waiting list the q applicants who rank highest, or all applicants if there are fewer than q , and rejects the rest. Rejected applicants then apply to their second choice and again each college selects the top q from among the new applicants and those on its waiting list, puts these on its new waiting list, and rejects the rest. The procedure terminates when every applicant is either on a waiting list or has been rejected by every college to which he is willing and permitted to apply. At this point each college

[form-feed in source extraction]

14 COLLEGE ADMISSIONS AND STABILITY OF MARRIAGE [January

admits everyone on its waiting list and the stable assignment has been achieved. The proof that the assignment is stable is entirely analogous to the proof given for the marriage problem and is left to the reader.

[Next verbatim source excerpt]

5. Optimality. We now show that the "deferred acceptance" procedure just described yields not only a stable but an optimal assignment of applicants. That is,

Lean-expanded paper semantic target

```
{Applicants : Type u_1} →
{Colleges : Type u_2} →
[inst : Fintype Colleges] →
[inst_1 : DecidableEq Applicants] →
(val_applicant : Applicants → Colleges → ℝ) →
(val_college : Colleges → Applicants → ℝ) →
(s : GS62CollegeAdmissions.ExactCollegeBatchedProcedure.SourceWaitingListState Applicants Colleges) →
(a : Applicants) →
{c |
  GS62CollegeAdmissions.ExactCollegeBatchedProcedure.sourceEligible val_applicant val_college a c ∧
  a ∉ s.applications c}
```

Recorded source-to-declaration screening Verdict: matches

Reason: The procedure continues only while an applicant has a college to which the applicant is willing and permitted to apply and has not already tried. The Lean set is that remaining choice set.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

--

GS62CollegeAdmissions.ExactCollegeBatchedProcedure.SourceActive

Paper-local declaration:

GS62CollegeAdmissions.ExactCollegeBatchedProcedure.SourceActive

Source locator: source.txt:232-249

Verbatim paper-source connection

4. Stable assignments and the admissions problem. The extension of our "deferred-acceptance" procedure to the problem of college admissions is straightforward. For convenience we will assume that if a college is not willing to accept a student under any circumstances, as described in Section 2, then that student will not even be permitted to apply to the college. With this understanding the procedure follows: First, all students apply to the college of their first choice. A college with a quota of q then places on its waiting list the q applicants who rank highest, or all applicants if there are fewer than q , and rejects the rest. Rejected applicants then apply to their second choice and again each college selects the top q from among the new applicants and those on its waiting list, puts these on its new waiting list, and rejects the rest. The procedure terminates when every applicant is either on a waiting list or has been rejected by every college to which he is willing and permitted to apply. At this point each college

[form-feed in source extraction]

14 COLLEGE ADMISSIONS AND STABILITY OF MARRIAGE [January

admits everyone on its waiting list and the stable assignment has been achieved. The proof that the assignment is stable is entirely analogous to the proof given for the marriage problem and is left to the reader.

Lean-expanded paper semantic target

```
∀ {Applicants : Type u_1} {Colleges : Type u_2} [inst : Fintype Colleges] [inst_1 : DecidableEq Applicants]
  (quota : Colleges → ℕ) (val_applicant : Applicants → Colleges → ℝ) (val_college : Colleges → Applicants → ℝ)
  (hcollegeStrict : ∀ (c : Colleges) (a a' : Applicants), val_college c a = val_college c a' → a = a')
  (s : GS62CollegeAdmissions.ExactCollegeBatchedProcedure.SourceWaitingListState Applicants Colleges) (a : Applicants),
  GS62CollegeAdmissions.ExactCollegeBatchedProcedure.assignedCollege quota val_college hcollegeStrict s a = none ∧
  (GS62CollegeAdmissions.ExactCollegeBatchedProcedure.untriedEligibleColleges val_applicant val_college s a).Nonempty
```

Recorded source-to-declaration screening Verdict: matches

Reason: The source stops when no applicant remains able to apply to a college that would consider them. The Lean predicate identifies that remaining active/applicable condition, so its negation is the source stopping condition used in the theorem.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GS62CollegeAdmissions.ExactCollegeBatchedProcedure.nextCollege

Paper-local declaration:

GS62CollegeAdmissions.ExactCollegeBatchedProcedure.nextCollege

Source locator: source.txt:232-249; source.txt:253-255 (printed pages 13-14, Sections 4-5 and Theorem 2)

Verbatim paper-source connection

4. Stable assignments and the admissions problem. The extension of our "deferred-acceptance" procedure to the problem of college admissions is straightforward. For convenience we will assume that if a college is not willing to accept a student under any circumstances, as described in Section 2, then that student will not even be permitted to apply to the college. With this understanding the procedure follows: First, all students apply to the college of their first choice. A college with a quota of q then places on its waiting list the q applicants who rank highest, or all applicants if there are fewer than q , and rejects the rest. Rejected applicants then apply to their second choice and again each college selects the top q from among the new applicants and those on its waiting list, puts these on its new waiting list, and rejects the rest. The procedure terminates when every applicant is either on a waiting list or has been rejected by every college to which he is willing and permitted to apply. At this point each college

[form-feed in source extraction]

14 COLLEGE ADMISSIONS AND STABILITY OF MARRIAGE [January

admits everyone on its waiting list and the stable assignment has been achieved. The proof that the assignment is stable is entirely analogous to the proof given for the marriage problem and is left to the reader.

[Next verbatim source excerpt]

THEOREM 2. Every applicant is at least as well off under the assignment given by the deferred acceptance procedure as he would be under any other stable assignment.

[Next verbatim source excerpt]

DEFINITION. An assignment of applicants to colleges will be called unstable if there are two applicants a and b who are assigned to colleges A and B , respectively, although A to b and A prefers: toa , b prefers A .

[Next verbatim source excerpt]

4. Stable assignments and the admissions problem. The extension of our "deferred-acceptance" procedure to the problem of college admissions is straightforward. For convenience we will assume that if a college is not willing to accept a student under any circumstances, as described in Section 2, then that student will not even be permitted to apply to the college. With this understanding the procedure follows: First, all students apply to the college of their first choice. A college with a quota of q then places on its waiting list the q applicants who rank highest, or all applicants if there are fewer than q , and rejects the rest. Rejected applicants then apply to their second choice and again each college selects the top q from among the new applicants and those on its waiting list, puts these on its new waiting list, and rejects the rest. The procedure terminates when every applicant is either on a waiting list or has been rejected by every college to which he is willing and permitted to apply. At this point each college

[form-feed in source extraction]

14 COLLEGE ADMISSIONS AND STABILITY OF MARRIAGE [January

admits everyone on its waiting list and the stable assignment has been achieved. The proof that the assignment is stable is entirely analogous to the proof given for the marriage problem and is left to the reader.

[Next verbatim source excerpt]

5. Optimality. We now show that the "deferred acceptance" procedure just described yields not only a stable but an optimal assignment of applicants. That is,

Lean-expanded paper semantic target

```
{Applicants : Type u_1} →
{Colleges : Type u_2} →
[inst : Fintype Applicants] →
[inst_1 : Fintype Colleges] →
[inst_2 : DecidableEq Applicants] →
[inst_3 : DecidableEq Colleges] →
(quota : Colleges → ℕ) →
(val_applicant : Applicants → Colleges → ℝ) →
(val_college : Colleges → Applicants → ℝ) →
(hcollegeStrict : ∀ (c : Colleges) (a a' : Applicants), val_college c a = val_college c a' → a = a') →
(s :
  GS62CollegeAdmissions.ExactCollegeBatchedProcedure.SourceWaitingListState Applicants Colleges) →
(a : Applicants) →
if hactive :
  GS62CollegeAdmissions.ExactCollegeBatchedProcedure.SourceActive quota val_applicant
  val_college hcollegeStrict s a then
  some (Classical.choose ...)
else none
```

Recorded source-to-declaration screening **Verdict:** matches

Reason: After rejection, the source directs an applicant to the next eligible choice. The Lean function selects that next untried eligible college in the applicant's preference order.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GS62CollegeAdmissions.ExactCollegeBatchedProcedure.newApplications

Paper-local declaration:

GS62CollegeAdmissions.ExactCollegeBatchedProcedure.newApplications

Source locator: source.txt:232-249; source.txt:253-255 (printed pages 13-14, Sections 4-5 and Theorem 2)

Verbatim paper-source connection

4. Stable assignments and the admissions problem. The extension of our "deferred-acceptance" procedure to the problem of college admissions is straightforward. For convenience we will assume that if a college is not willing to accept a student under any circumstances, as described in Section 2, then that student will not even be permitted to apply to the college. With this understanding the procedure follows: First, all students apply to the college of their first choice. A college with a quota of q then places on its waiting list the q applicants who rank highest, or all applicants if there are fewer than q , and rejects the rest. Rejected applicants then apply to their second choice and again each college selects the top q from among the new applicants and those on its waiting list, puts these on its new waiting list, and rejects the rest. The procedure terminates when every applicant is either on a waiting list or has been rejected by every college to which he is willing and permitted to apply. At this point each college

[form-feed in source extraction]

14 COLLEGE ADMISSIONS AND STABILITY OF MARRIAGE [January

admits everyone on its waiting list and the stable assignment has been achieved. The proof that the assignment is stable is entirely analogous to the proof given for the marriage problem and is left to the reader.

[Next verbatim source excerpt]

THEOREM 2. Every applicant is at least as well off under the assignment given by the deferred acceptance procedure as he would be under any other stable assignment.

[Next verbatim source excerpt]

DEFINITION. An assignment of applicants to colleges will be called unstable if there are two applicants a and b who are assigned to colleges A and B , respectively, although A to b and A prefers: $to a$.
 b prefers

[Next verbatim source excerpt]

4. Stable assignments and the admissions problem. The extension of our "deferred-acceptance" procedure to the problem of college admissions is straightforward. For convenience we will assume that if a college is not willing to accept a student under any circumstances, as described in Section 2, then that student will not even be permitted to apply to the college. With this understanding the procedure follows: First, all students apply to the college of their first choice. A college with a quota of q then places on its waiting list the q applicants who rank highest, or all applicants if there are fewer than q , and rejects the rest. Rejected applicants then apply to their second choice and again each college selects the top q from among the new applicants and those on its waiting list, puts these on its new waiting list, and rejects the rest. The procedure terminates when every applicant is either on a waiting list or has been rejected by every college to which he is willing and permitted to apply. At this point each college

[form-feed in source extraction]

14 COLLEGE ADMISSIONS AND STABILITY OF MARRIAGE [January

admits everyone on its waiting list and the stable assignment has been achieved. The proof that the assignment is stable is entirely analogous to the proof given for the marriage problem and is left to the reader.

[Next verbatim source excerpt]

5. Optimality. We now show that the "deferred acceptance" procedure just described yields not only a stable but an optimal assignment of applicants. That is,

Lean-expanded paper semantic target

```
{Applicants : Type u_1} →
{Colleges : Type u_2} →
[inst : Fintype Applicants] →
[inst_1 : Fintype Colleges] →
[inst_2 : DecidableEq Applicants] →
[inst_3 : DecidableEq Colleges] →
(quota : Colleges → ℕ) →
(val_applicant : Applicants → Colleges → ℝ) →
(val_college : Colleges → Applicants → ℝ) →
(hcollegeStrict : ∀ (c : Colleges) (a a' : Applicants), val_college c a = val_college c a' → a = a') →
(s :
  GS62CollegeAdmissions.ExactCollegeBatchedProcedure.SourceWaitingListState Applicants Colleges) →
(c : Colleges) →
{a |
  GS62CollegeAdmissions.ExactCollegeBatchedProcedure.nextCollege quota val_applicant val_college
    hcollegeStrict s a =
  some c}
```

Recorded source-to-declaration screening Verdict: matches

Reason: The source sends each rejected applicant to the next college on that applicant's list that is willing and permitted to consider the applicant. The Lean function collects exactly those round applications.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GS62CollegeAdmissions.ExactCollegeBatchedProcedure.sourceStep

Paper-local declaration:

GS62CollegeAdmissions.ExactCollegeBatchedProcedure.sourceStep

Source locator: source.txt:232-249; source.txt:253-255 (printed pages 13-14, Sections 4-5 and Theorem 2)

Verbatim paper-source connection

4. Stable assignments and the admissions problem. The extension of our "deferred-acceptance" procedure to the problem of college admissions is straightforward. For convenience we will assume that if a college is not willing to accept a student under any circumstances, as described in Section 2, then that student will not even be permitted to apply to the college. With this understanding the procedure follows: First, all students apply to the college of their first choice. A college with a quota of q then places on its waiting list the q applicants who rank highest, or all applicants if there are fewer than q , and rejects the rest. Rejected applicants then apply to their second choice and again each college selects the top q from among the new applicants and those on its waiting list, puts these on its new waiting list, and rejects the rest. The procedure terminates when every applicant is either on a waiting list or has been rejected by every college to which he is willing and permitted to apply. At this point each college

[form-feed in source extraction]

14 COLLEGE ADMISSIONS AND STABILITY OF MARRIAGE [January

admits everyone on its waiting list and the stable assignment has been achieved. The proof that the assignment is stable is entirely analogous to the proof given for the marriage problem and is left to the reader.

[Next verbatim source excerpt]

THEOREM 2. Every applicant is at least as well off under the assignment given by the deferred acceptance procedure as he would be under any other stable assignment.

[Next verbatim source excerpt]

DEFINITION. An assignment of applicants to colleges will be called unstable if there are two applicants a and b who are assigned to colleges A and B , respectively, although A to b and A prefers: $to a$.
 b prefers

[Next verbatim source excerpt]

4. Stable assignments and the admissions problem. The extension of our "deferred-acceptance" procedure to the problem of college admissions is straightforward. For convenience we will assume that if a college is not willing to accept a student under any circumstances, as described in Section 2, then that student will not even be permitted to apply to the college. With this understanding the procedure follows: First, all students apply to the college of their first choice. A college with a quota of q then places on its waiting list the q applicants who rank highest, or all applicants if there are fewer than q , and rejects the rest. Rejected applicants then apply to their second choice and again each college selects the top q from among the new applicants and those on its waiting list, puts these on its new waiting list, and rejects the rest. The procedure terminates when every applicant is either on a waiting list or has been rejected by every college to which he is willing and permitted to apply. At this point each college

[form-feed in source extraction]

14 COLLEGE ADMISSIONS AND STABILITY OF MARRIAGE [January

admits everyone on its waiting list and the stable assignment has been achieved. The proof that the assignment is stable is entirely analogous to the proof given for the marriage problem and is left to the reader.

[Next verbatim source excerpt]

5. Optimality. We now show that the "deferred acceptance" procedure just described yields not only a stable but an optimal assignment of applicants. That is,

Lean-expanded paper semantic target

```
{Applicants : Type u_1} →
{Colleges : Type u_2} →
[inst : Fintype Applicants] →
[inst_1 : Fintype Colleges] →
[inst_2 : DecidableEq Applicants] →
[inst_3 : DecidableEq Colleges] →
(quota : Colleges → ℕ) →
(val_applicant : Applicants → Colleges → ℝ) →
(val_college : Colleges → Applicants → ℝ) →
(hcollegeStrict : ∀ (c : Colleges) (a a' : Applicants), val_college c a = val_college c a' → a = a') →
(s :
  GS62CollegeAdmissions.ExactCollegeBatchedProcedure.SourceWaitingListState Applicants Colleges) →
{
  applications := fun c =>
    s.applications c u
    GS62CollegeAdmissions.ExactCollegeBatchedProcedure.newApplications quota val_applicant
    val_college hcollegeStrict s c }
```

Recorded source-to-declaration screening Verdict: matches

Reason: A source round combines the newly received applications with the old waiting list and retains the top q while rejecting the rest. The Lean function is that simultaneous update.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GS62CollegeAdmissions.ExactCollegeBatchedProcedure.SourceStepRelation

Paper-local declaration:

GS62CollegeAdmissions.ExactCollegeBatchedProcedure.SourceStepRelation

Source locator: source.txt:232-249; source.txt:253-255 (printed pages 13-14, Sections 4-5 and Theorem 2)

Verbatim paper-source connection

4. Stable assignments and the admissions problem. The extension of our "deferred-acceptance" procedure to the problem of college admissions is straightforward. For convenience we will assume that if a college is not willing to accept a student under any circumstances, as described in Section 2, then that student will not even be permitted to apply to the college. With this understanding the procedure follows: First, all students apply to the college of their first choice. A college with a quota of q then places on its waiting list the q applicants who rank highest, or all applicants if there are fewer than q , and rejects the rest. Rejected applicants then apply to their second choice and again each college selects the top q from among the new applicants and those on its waiting list, puts these on its new waiting list, and rejects the rest. The procedure terminates when every applicant is either on a waiting list or has been rejected by every college to which he is willing and permitted to apply. At this point each college

[form-feed in source extraction]

14 COLLEGE ADMISSIONS AND STABILITY OF MARRIAGE [January

admits everyone on its waiting list and the stable assignment has been achieved. The proof that the assignment is stable is entirely analogous to the proof given for the marriage problem and is left to the reader.

[Next verbatim source excerpt]

THEOREM 2. Every applicant is at least as well off under the assignment given by the deferred acceptance procedure as he would be under any other stable assignment.

[Next verbatim source excerpt]

DEFINITION. An assignment of applicants to colleges will be called unstable if there are two applicants a and b who are assigned to colleges A and B , respectively, although A to b and A prefers: toa .
 b prefers

[Next verbatim source excerpt]

4. Stable assignments and the admissions problem. The extension of our "deferred-acceptance" procedure to the problem of college admissions is straightforward. For convenience we will assume that if a college is not willing to accept a student under any circumstances, as described in Section 2, then that student will not even be permitted to apply to the college. With this understanding the procedure follows: First, all students apply to the college of their first choice. A college with a quota of q then places on its waiting list the q applicants who rank highest, or all applicants if there are fewer than q , and rejects the rest. Rejected applicants then apply to their second choice and again each college selects the top q from among the new applicants and those on its waiting list, puts these on its new waiting list, and rejects the rest. The procedure terminates when every applicant is either on a waiting list or has been rejected by every college to which he is willing and permitted to apply. At this point each college

[form-feed in source extraction]

14 COLLEGE ADMISSIONS AND STABILITY OF MARRIAGE [January

admits everyone on its waiting list and the stable assignment has been achieved. The proof that the assignment is stable is entirely analogous to the proof given for the marriage problem and is left to the reader.

[Next verbatim source excerpt]

5. Optimality. We now show that the "deferred acceptance" procedure just described yields not only a stable but an optimal assignment of applicants. That is,

Lean-expanded paper semantic target

```
∀ {Applicants : Type u_1} {Colleges : Type u_2} [inst : Fintype Applicants] [inst_1 : Fintype Colleges]
[inst_2 : DecidableEq Applicants] [inst_3 : DecidableEq Colleges] (quota : Colleges → ℕ)
(val_applicant : Applicants → Colleges → ℝ) (val_college : Colleges → Applicants → ℝ)
(hcollegeStrict : ∀ (c : Colleges) (a a' : Applicants), val_college c a = val_college c a' → a = a')
(before after : GS62CollegeAdmissions.ExactCollegeBatchedProcedure.SourceWaitingListState Applicants Colleges),
after =
  GS62CollegeAdmissions.ExactCollegeBatchedProcedure.sourceStep quota val_applicant val_college hcollegeStrict before
```

Recorded source-to-declaration screening Verdict: matches

Reason: The source proceeds by repeated batches of applications, quota-sized waiting-list selection, and rejection. The Lean relation is exactly one such source procedure round.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GS62CollegeAdmissions.ExactCollegeBatchedProcedure.initialSourceState

Paper-local declaration:

GS62CollegeAdmissions.ExactCollegeBatchedProcedure.initialSourceState

Source locator: source.txt:232-249; source.txt:253-255 (printed pages 13-14, Sections 4-5 and Theorem 2)

Verbatim paper-source connection

4. Stable assignments and the admissions problem. The extension of our "deferred-acceptance" procedure to the problem of college admissions is straightforward. For convenience we will assume that if a college is not willing to accept a student under any circumstances, as described in Section 2, then that student will not even be permitted to apply to the college. With this understanding the procedure follows: First, all students apply to the college of their first choice. A college with a quota of q then places on its waiting list the q applicants who rank highest, or all applicants if there are fewer than q , and rejects the rest. Rejected applicants then apply to their second choice and again each college selects the top q from among the new applicants and those on its waiting list, puts these on its new waiting list, and rejects the rest. The procedure terminates when every applicant is either on a waiting list or has been rejected by every college to which he is willing and permitted to apply. At this point each college

[form-feed in source extraction]
14 COLLEGE ADMISSIONS AND STABILITY OF MARRIAGE [January

admits everyone on its waiting list and the stable assignment has been achieved. The proof that the assignment is stable is entirely analogous to the proof given for the marriage problem and is left to the reader.

[Next verbatim source excerpt]

THEOREM 2. Every applicant is at least as well off under the assignment given by the deferred acceptance procedure as he would be under any other stable assignment.

[Next verbatim source excerpt]

DEFINITION. An assignment of applicants to colleges will be called unstable if there are two applicants a and b who are assigned to colleges A and B , respectively, although A to b and A prefers: toa .
 a prefers b .

[Next verbatim source excerpt]

4. Stable assignments and the admissions problem. The extension of our "deferred-acceptance" procedure to the problem of college admissions is straightforward. For convenience we will assume that if a college is not willing to accept a student under any circumstances, as described in Section 2, then that student will not even be permitted to apply to the college. With this understanding the procedure follows: First, all students apply to the college of their first choice. A college with a quota of q then places on its waiting list the q applicants who rank highest, or all applicants if there are fewer than q , and rejects the rest. Rejected applicants then apply to their second choice and again each college selects the top q from among the new applicants and those on its waiting list, puts these on its new waiting list, and rejects the rest. The procedure terminates when every applicant is either on a waiting list or has been rejected by every college to which he is willing and permitted to apply. At this point each college

[form-feed in source extraction]
14 COLLEGE ADMISSIONS AND STABILITY OF MARRIAGE [January

admits everyone on its waiting list and the stable assignment has been achieved. The proof that the assignment is stable is entirely analogous to the proof given for the marriage problem and is left to the reader.

[Next verbatim source excerpt]

5. Optimality. We now show that the "deferred acceptance" procedure just described yields not only a stable but an optimal assignment of applicants. That is,

Lean-expanded paper semantic target

{Applicants : Type u₁} → {Colleges : Type u₂} → [inst : DecidableEq Applicants] → { applications := fun x => ∅ }

Recorded source-to-declaration screening Verdict: matches

Reason: Before the first applications, no college has yet retained applicants. The Lean state is the empty-waiting-list initial state from which the stated procedure starts.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GS62CollegeAdmissions.ExactCollegeBatchedProcedure.SourceReachable

Paper-local declaration:

GS62CollegeAdmissions.ExactCollegeBatchedProcedure.SourceReachable

Source locator: source.txt:232-249

Verbatim paper-source connection

4. Stable assignments and the admissions problem. The extension of our "deferred-acceptance" procedure to the problem of college admissions is straightforward. For convenience we will assume that if a college is not willing to accept a student under any circumstances, as described in Section 2, then that student will not even be permitted to apply to the college. With this understanding the procedure follows: First, all students apply to the college of their first choice. A college with a quota of q then places on its waiting list the q applicants who rank highest, or all applicants if there are fewer than q , and rejects the rest. Rejected applicants then apply to their second choice and again each college selects the top q from among the new applicants and those on its waiting list, puts these on its new waiting list, and rejects the rest. The procedure terminates when every applicant is either on a waiting list or has been rejected by every college to which he is willing and permitted to apply. At this point each college

[form-feed in source extraction]

14 COLLEGE ADMISSIONS AND STABILITY OF MARRIAGE [January

admits everyone on its waiting list and the stable assignment has been achieved. The proof that the assignment is stable is entirely analogous to the proof given for the marriage problem and is left to the reader.

Lean-expanded paper semantic target

```
∀ {Applicants : Type u_1} {Colleges : Type u_2} [inst : Fintype Applicants] [inst_1 : Fintype Colleges]
  [inst_2 : DecidableEq Applicants] [inst_3 : DecidableEq Colleges] (quota : Colleges → ℕ)
  (val_applicant : Applicants → Colleges → ℝ) (val_college : Colleges → Applicants → ℝ)
  (hcollegeStrict : ∀ (c : Colleges) (a a' : Applicants), val_college c a = val_college c a' → a = a')
  (s : GS62CollegeAdmissions.ExactCollegeBatchedProcedure.SourceWaitingListState Applicants Colleges),
  Relation.ReflTransGen
    (GS62CollegeAdmissions.ExactCollegeBatchedProcedure.SourceStepRelation quota val_applicant val_college
     hcollegeStrict)
  GS62CollegeAdmissions.ExactCollegeBatchedProcedure.initialSourceState s
```

Recorded source-to-declaration screening Verdict: matches

Reason: The source defines the college procedure by successive application and waiting-list rounds from the empty initial state. The Lean predicate identifies exactly the states reachable by those rounds; it does not introduce an externally supplied terminal trace.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GS62CollegeAdmissions.ExactCollegeBatchedProcedure.sourceAssignmentView

Paper-local declaration:

GS62CollegeAdmissions.ExactCollegeBatchedProcedure.sourceAssignmentView

Source locator: source.txt:232-249

Verbatim paper-source connection

4. Stable assignments and the admissions problem. The extension of our "deferred-acceptance" procedure to the problem of college admissions is straightforward. For convenience we will assume that if a college is not willing to accept a student under any circumstances, as described in Section 2, then that student will not even be permitted to apply to the college. With this understanding the procedure follows: First, all students apply to the college of their first choice. A college with a quota of q then places on its waiting list the q applicants who rank highest, or all applicants if there are fewer than q , and rejects the rest. Rejected applicants then apply to their second choice and again each college selects the top q from among the new applicants and those on its waiting list, puts these on its new waiting list, and rejects the rest. The procedure terminates when every applicant is either on a waiting list or has been rejected by every college to which he is willing and permitted to apply. At this point each college

[form-feed in source extraction]

14 COLLEGE ADMISSIONS AND STABILITY OF MARRIAGE [January

admits everyone on its waiting list and the stable assignment has been achieved. The proof that the assignment is stable is entirely analogous to the proof given for the marriage problem and is left to the reader.

Lean-expanded paper semantic target

```
{Applicants : Type u_1} →
{Colleges : Type u_2} →
  [inst : Fintype Applicants] →
  [inst_1 : Fintype Colleges] →
  [inst_2 : DecidableEq Applicants] →
  [inst_3 : DecidableEq Colleges] →
  (quota : Colleges → ℕ) →
  (val_college : Colleges → Applicants → ℝ) →
  (hcollegeStrict : ∀ (c : Colleges) (a a' : Applicants), val_college c a = val_college c a' → a = a') →
  (s : GS62CollegeAdmissions.ExactCollegeBatchedProcedure.SourceWaitingListState Applicants Colleges) →
  {
    app_match :=
      GS62CollegeAdmissions.ExactCollegeBatchedProcedure.assignedCollege quota val_college
      hcollegeStrict s,
    college_roster := fun c =>
      {a |
        GS62CollegeAdmissions.ExactCollegeBatchedProcedure.assignedCollege quota val_college
        hcollegeStrict s a =
        some c},
    consistent := ... }
}
```

Recorded source-to-declaration screening Verdict: matches

Reason: The source assigns each applicant to the college whose final waiting list contains them, and leaves otherwise rejected applicants unassigned. The Lean view reads exactly that assignment from the terminal waiting-list state.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GS62CollegeAdmissions.gs62ReplacementPair

Paper-local declaration:

GS62CollegeAdmissions.gs62ReplacementPair

Source locator: source.txt:93-98 (printed page 10, first displayed Definition)

Verbatim paper-source connection

DEFINITION. An assignment of applicantstocolleges willbecalledunstableif thereare twoapplicantsa and ,Bwhoare assignedtocolleges A andB, respectively, although A toB and A prefers: toa. ,3prefers

Lean-expanded paper semantic target

```
∀ {Applicants : Type u_1} {Colleges : Type u_2} (val_applicant : Applicants → Colleges → ℝ)
  (val_college : Colleges → Applicants → ℝ) (mu : EconCSLib.Matching.ManyToOneAssignment Applicants Colleges),
  ∃ alpha beta A B,
    mu.app_match alpha = some A ∧
    mu.app_match beta = some B ∧
    val_applicant beta B < val_applicant beta A ∧ val_college A alpha < val_college A beta
```

Recorded source-to-declaration screening Verdict: matches

Reason: The source definition of an unstable college assignment is the existence of the stated applicant-college replacement pair. The Lean predicate gives exactly those pair conditions.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GS62CollegeAdmissions.gs62LiteralStableCollegeAssignment

Paper-local declaration:

GS62CollegeAdmissions.gs62LiteralStableCollegeAssignment

Source locator: source.txt:93-98 (printed page 10, first displayed Definition)

Verbatim paper-source connection

DEFINITION. An assignment of applicantstocolleges willbecalledunstableif thereare twoapplicantsa and ,Bwhoare assignedtocolleges A andB, respectively, although A toB and A prefers: toa. ,3prefers

Lean-expanded paper semantic target

```
∀ {Applicants : Type u_1} {Colleges : Type u_2} (val_applicant : Applicants → Colleges → ℝ)
  (val_college : Colleges → Applicants → ℝ) (mu : EconCSLib.Matching.ManyToOneAssignment Applicants Colleges),
  ~GS62CollegeAdmissions.gs62ReplacementPair val_applicant val_college mu
```

Recorded source-to-declaration screening Verdict: matches

Reason: The source calls a college assignment stable exactly when it has no replacement-pair instability. The Lean predicate is the corresponding negation of that source definition.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GS62CollegeAdmissions.gs62LiteralApplicantOptimalCollegeAssignment

Paper-local declaration:

GS62CollegeAdmissions.gs62LiteralApplicantOptimalCollegeAssignment

Source locator: source.txt:115-117 (printed page 10, second displayed Definition)

Verbatim paper-source connection

DEFINITION. A stableassignment μ is called optimal if every applicant a is at least as well off under μ as under any other stable assignment.

Lean-expanded paper semantic target

```
∀ {Applicants : Type u_1} {Colleges : Type u_2} (val_applicant : Applicants → Colleges → ℝ)
  (val_college : Colleges → Applicants → ℝ) (mu : EconCSLib.Matching.ManyToOneAssignment Applicants Colleges),
  GS62CollegeAdmissions.gs62LiteralStableCollegeAssignment val_applicant val_college mu ∧
  ∀ (nu : EconCSLib.Matching.ManyToOneAssignment Applicants Colleges),
    GS62CollegeAdmissions.gs62LiteralStableCollegeAssignment val_applicant val_college nu →
    ∀ (a : Applicants),
      EconCSLib.Matching.ManyToOne.valApplicant val_applicant a (nu.app_match a) ≤
      EconCSLib.Matching.ManyToOne.valApplicant val_applicant a (mu.app_match a)
```

Recorded source-to-declaration screening Verdict: matches

Reason: The source calls an assignment optimal for applicants when every applicant weakly prefers it to every other stable assignment. The Lean predicate is that literal comparison.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GS62CollegeAdmissions.PaperInterface.literalApplicantOptimalCollegeAssignment

Paper-local declaration:

GS62CollegeAdmissions.PaperInterface.literalApplicantOptimalCollegeAssignment

Source locator: source.txt:115-117 (printed page 10, second displayed Definition)

Verbatim paper-source connection

DEFINITION. A stableassignment μ is called optimal if every applicant a is at least as well off under μ as under any other stable assignment.

Lean-expanded paper semantic target

```
∀ {Applicants : Type u_1} {Colleges : Type u_2} (val_applicant : Applicants → Colleges → ℝ)
  (val_college : Colleges → Applicants → ℝ) (mu : EconCSLib.Matching.ManyToOneAssignment Applicants Colleges),
  GS62CollegeAdmissions.gs62LiteralApplicantOptimalCollegeAssignment val_applicant val_college mu
```

Recorded source-to-declaration screening Verdict: matches

Reason: The source's second displayed definition selects, among its literal stable assignments, one that every applicant weakly prefers to every other such assignment. The Lean declaration writes that source comparison class and applicant-by-applicant ordering directly.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GS62CollegeAdmissions.PaperInterface.replacementPairCollegeInstability

Paper-local declaration:

GS62CollegeAdmissions.PaperInterface.replacementPairCollegeInstability

Source locator: source.txt:93-98 (printed page 10, first displayed Definition)

Verbatim paper-source connection

DEFINITION. An assignment of applicantstocolleges willbecalledunstableif thereare twoapplicantsa and ,Bwhoare assignedtocolleges A andB, respectively, although A toB and A prefers: toa. ,3prefers

Lean-expanded paper semantic target

```
∀ {Applicants : Type u_1} {Colleges : Type u_2} (val_applicant : Applicants → Colleges → ℝ)
  (val_college : Colleges → Applicants → ℝ) (mu : EconCSLib.Matching.ManyToOneAssignment Applicants Colleges),
  GS62CollegeAdmissions.gs62ReplacementPair val_applicant val_college mu
```

Recorded source-to-declaration screening Verdict: matches

Reason: The displayed source definition identifies instability through an applicant who prefers another college and a replacement at that college whom it ranks lower. The Lean predicate is that replacement pair.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GS62CollegeAdmissions.PaperInterface.unstableCollegeAssignment

Paper-local declaration:

GS62CollegeAdmissions.PaperInterface.unstableCollegeAssignment

Source locator: source.txt:93-98 (printed page 10, first displayed Definition)

Verbatim paper-source connection

DEFINITION. An assignment of applicantstocolleges willbecalledunstableif thereare twoapplicantsta and ,Bwhoare assignedtocolleges A andB, respectively, although A toB and A prefers: toa. ,3prefers

Lean-expanded paper semantic target

```
∀ {Applicants : Type u_1} {Colleges : Type u_2} (val_applicant : Applicants → Colleges → ℝ)
  (val_college : Colleges → Applicants → ℝ) (mu : EconCSLib.Matching.ManyToOneAssignment Applicants Colleges),
  GS62CollegeAdmissions.PaperInterface.replacementPairCollegeInstability val_applicant val_college mu
```

Recorded source-to-declaration screening Verdict: matches

Reason: The source's first displayed college definition is the existence of two currently assigned applicants and colleges for which one applicant prefers the other college and that college prefers the replacement. The Lean predicate is exactly that replacement-pair condition.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GS62CollegeAdmissions.PaperInterface.unstableMarriage

Paper-local declaration:

GS62CollegeAdmissions.PaperInterface.unstableMarriage

Source locator: source.txt:133-139 (printed page 11, Section 3)

Verbatim paper-source connection

A certain community consists of n men and n women. Each person ranks those of the opposite sex in accordance with his or her preferences for a marriage partner. We seek a satisfactory way of marrying off all members of the community. Imitating our earlier definition, we call a set of marriages unstable (and here the suitability of the term is quite clear) if under it there are a man and a woman who are not married to each other but prefer each other to their actual mates.

Lean-expanded paper semantic target

```
∀ {M : Type u_1} {W : Type u_2} (val_m : M → W → ℝ) (val_w : W → M → ℝ) (mu : EconCSLib.Matching.Assignment M W),
  ((∀ (m : M), ∃ w, mu.m_match m = some w) ∧ ∀ (w : W), ∃ m, mu.w_match w = some m) ∧
  ∃ m w,
    EconCSLib.Matching.valM val_m m (mu.m_match m) < val_m m w ∧
    EconCSLib.Matching.valW val_w w (mu.w_match w) < val_w w m
```

Recorded source-to-declaration screening Verdict: matches

Reason: The source calls a complete marriage unstable when some man and woman each prefer one another to their current partners. The Lean predicate records the same complete matching and mutual strict improvement condition.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Main-text source claims

Review row 1

Source locator: source.txt:93-98 (printed page 10, first displayed Definition)

Verbatim source input

DEFINITION. An assignment of applicants to colleges will be called unstable if there are two applicants a and b who are assigned to colleges A and B , respectively, although a prefers B to A and A prefers a to b .

Expanded PaperInterface specification

```
∀ {Applicants : Type u_1} {Colleges : Type u_2} (val_applicant : Applicants → Colleges → ℝ)
  (val_college : Colleges → Applicants → ℝ) (mu : EconCSLib.Matching.ManyToOneAssignment Applicants Colleges),
  GS62CollegeAdmissions.PaperInterface.unstableCollegeAssignment val_applicant val_college mu ↔
    ∃ alpha beta A B,
      mu.app_match alpha = some A ∧
      mu.app_match beta = some B ∧
      val_applicant beta B < val_applicant beta A ∧ val_college A alpha < val_college A beta
```

Lean proof endpoint (built separately)

GS62CollegeAdmissions.ProofInterface.unstableCollegeAssignment_iff_source_definition

Recorded source-to-Spec screening Verdict: matches

Reason: The transparent Spec states the displayed current-assignment replacement pair directly: two currently assigned applicants, their current colleges, the applicant's strict improvement, and the college's strict preference for the replacement applicant. It adds no vacancy, unmatched-applicant, or individual-rationality condition.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 2

Source locator: source.txt:115-117 (printed page 10, second displayed Definition)

Verbatim source input

DEFINITION. A stableassignment μ is called optimal if every applicant a is at least as well off under μ as under any other stable assignment.

Expanded PaperInterface specification

```

∀ {Applicants : Type u_1} {Colleges : Type u_2} (val_applicant : Applicants → Colleges → ℝ)
  (val_college : Colleges → Applicants → ℝ) (mu : EconCSLib.Matching.ManyToOneAssignment Applicants Colleges),
  GS62CollegeAdmissions.PaperInterface.literalApplicantOptimalCollegeAssignment val_applicant val_college mu ↔
  (¬∃ alpha beta A B,
    mu.app_match alpha = some A ∧
    mu.app_match beta = some B ∧
    val_applicant beta B < val_applicant beta A ∧ val_college A alpha < val_college A beta) ∧
  ∀ (nu : EconCSLib.Matching.ManyToOneAssignment Applicants Colleges),
  (¬∃ alpha beta A B,
    nu.app_match alpha = some A ∧
    nu.app_match beta = some B ∧
    val_applicant beta B < val_applicant beta A ∧ val_college A alpha < val_college A beta) →
  ∀ (a : Applicants),
  (match nu.app_match a with
  | none => 0
  | some c => val_applicant a c) ≤
  (match mu.app_match a with
  | none => 0
  | some c => val_applicant a c)

```

Lean proof endpoint (built separately)

GS62CollegeAdmissions.ProofInterface.literalApplicantOptimalCollegeAssignment_iff_source_definition

Recorded source-to-Spec screening Verdict: matches

Reason: The transparent Spec preserves the literal page-10 comparison class: quota-feasible assignments with no displayed replacement pair and applicant-by-applicant weak preference for the selected assignment. This row intentionally does not use the Section 4 operational stability convention.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 3

Source locator: source.txt:133-139 (printed page 11, Section 3)

Verbatim source input

A certain community consists of n men and n women. Each person ranks those of the opposite sex in accordance with his or her preferences for a marriage partner. We seek a satisfactory way of marrying off all members of the community. Imitating our earlier definition, we call a set of marriages unstable (and here the suitability of the term is quite clear) if under it there are a man and a woman who are not married to each other but prefer each other to their actual mates.

Expanded PaperInterface specification

```
∀ {M : Type u_1} {W : Type u_2} (val_m : M → W → ℝ) (val_w : W → M → ℝ) (mu : EconCSLib.Matching.Assignment M W),
  GS62CollegeAdmissions.PaperInterface.unstableMarriage val_m val_w mu ↔
  ((∀ (m : M), ∃ w, mu.m_match m = some w) ∧ ∀ (w : W), ∃ m, mu.w_match w = some m) ∧
  ∃ m w,
    EconCSLib.Matching.valM val_m m (mu.m_match m) < val_m m w ∧
    EconCSLib.Matching.valW val_w w (mu.w_match w) < val_w w m
```

Lean proof endpoint (built separately)

GS62CollegeAdmissions.ProofInterface.unstableMarriage_iff_source_definition

Recorded source-to-Spec screening Verdict: matches

Reason: The source's marriage discussion concerns a complete matching and a man-woman pair who each prefer the other to the present partner. The transparent Spec writes the completeness and both strict present-partner comparisons explicitly.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 4

Source locator: source.txt:127-139; source.txt:185 (printed pages 11-12, Section 3 and Theorem 1)

Verbatim source input

3. Stable assignments and a marriage problem. In trying to settle the question of the existence of stable assignments we were led to look first at a special case, in which there are the same number of applicants as colleges and all quotas are unity. This situation is, of course, highly unnatural in the context of college admissions, but there is another "story" into which it fits quite readily.

A certain community consists of n men and n women. Each person ranks those of the opposite sex in accordance with his or her preferences for a marriage partner. We seek a satisfactory way of marrying off all members of the community. Imitating our earlier definition, we call a set of marriages unstable (and here the suitability of the term is quite clear) if under it there are a man and a woman who are not married to each other but prefer each other to their actual mates.

[Next verbatim source excerpt]

THEOREM 1. There always exists a stable set of marriages.

[Next verbatim source excerpt]

3. Stable assignments and a marriage problem. In trying to settle the question of the existence of stable assignments we were led to look first at a special case, in which there are the same number of applicants as colleges and all quotas are unity. This situation is, of course, highly unnatural in the context of college admissions, but there is another "story" into which it fits quite readily.

[Next verbatim source excerpt]

A certain community consists of n men and n women. Each person ranks those of the opposite sex in accordance with his or her preferences for a marriage partner. We seek a satisfactory way of marrying off all members of the community. Imitating our earlier definition, we call a set of marriages unstable (and here the suitability of the term is quite clear) if under it there are a man and a woman who are not married to each other but prefer each other to their actual mates.

Expanded PaperInterface specification

```
∀ {M : Type u_1} {W : Type u_2} [inst : Fintype M] [inst_1 : Fintype W] [DecidableEq M] [DecidableEq W]
  (val_m : M → W → ℝ) (val_w : W → M → ℝ),
  Fintype.card M = Fintype.card W →
  (∀ (m : M) (w w' : W), val_m m w = val_m m w' → w = w') →
  (∀ (w : W) (m m' : M), val_w w m = val_w w m' → m = m') →
  (∀ (m : M) (w : W), 0 < val_m m w) →
  (∀ (w : W) (m : M), 0 < val_w w m) →
  ∃ mu,
  ¬((∀ (m : M), ∃ w, mu.m_match m = some w) ∧ ∀ (w : W), ∃ m, mu.w_match w = some m) ∧
  ∃ m w,
  EconCSLib.Matching.valM val_m m (mu.m_match m) < val_m m w ∧
  EconCSLib.Matching.valW val_w w (mu.w_match w) < val_w w m) ∧
  (∀ (m : M), ∃ w, mu.m_match m = some w) ∧ ∀ (w : W), ∃ m, mu.w_match w = some m
```

Lean proof endpoint (built separately)

GS62CollegeAdmissions.ProofInterface.theorem1_stable_marriage_exists

Recorded source-to-Spec screening Verdict: matches

Reason: The transparent Spec has the source's finite equal-size strict complete domain and concludes a complete assignment with no mutually preferred blocking pair. Its expanded form makes the source's stable-marriage claim checkable without a wrapper predicate.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 5

Source locator: source.txt:232-249; source.txt:253-255 (printed pages 13-14, Sections 4-5 and Theorem 2)

Verbatim source input

4. Stable assignments and the admissions problem. The extension of our "deferred-acceptance" procedure to the problem of college admissions is straightforward. For convenience we will assume that if a college is not willing to accept a student under any circumstances, as described in Section 2, then that student will not even be permitted to apply to the college. With this understanding the procedure follows: First, all students apply to the college of their first choice. A college with a quota of q then places on its waiting list the q applicants who rank highest, or all applicants if there are fewer than q , and rejects the rest. Rejected applicants then apply to their second choice and again each college selects the top q from among the new applicants and those on its waiting list, puts these on its new waiting list, and rejects the rest. The procedure terminates when every applicant is either on a waiting list or has been rejected by every college to which he is willing and permitted to apply. At this point each college

[form-feed in source extraction]
14 COLLEGE ADMISSIONS AND STABILITY OF MARRIAGE [January

admits everyone on its waiting list and the stable assignment has been achieved. The proof that the assignment is stable is entirely analogous to the proof given for the marriage problem and is left to the reader.

[Next verbatim source excerpt]

THEOREM 2. Every applicant is at least as well off under the assignment given by the deferred acceptance procedure as he would be under any other stable assignment.

[Next verbatim source excerpt]

DEFINITION. An assignment of applicants to colleges will be called unstable if there are two applicants a and b who are assigned to colleges A and B , respectively, although A to B and A prefers: $to a$.
, 3 prefers

[Next verbatim source excerpt]

4. Stable assignments and the admissions problem. The extension of our "deferred-acceptance" procedure to the problem of college admissions is straightforward. For convenience we will assume that if a college is not willing to accept a student under any circumstances, as described in Section 2, then that student will not even be permitted to apply to the college. With this understanding the procedure follows: First, all students apply to the college of their first choice. A college with a quota of q then places on its waiting list the q applicants who rank highest, or all applicants if there are fewer than q , and rejects the rest. Rejected applicants then apply to their second choice and again each college selects the top q from among the new applicants and those on its waiting list, puts these on its new waiting list, and rejects the rest. The procedure terminates when every applicant is either on a waiting list or has been rejected by every college to which he is willing and permitted to apply. At this point each college

[form-feed in source extraction]
14 COLLEGE ADMISSIONS AND STABILITY OF MARRIAGE [January

admits everyone on its waiting list and the stable assignment has been achieved. The proof that the assignment is stable is entirely analogous to the proof given for the marriage problem and is left to the reader.

[Next verbatim source excerpt]

5. Optimality. We now show that the "deferred acceptance" procedure just described yields not only a stable but an optimal assignment of applicants. That is,

Expanded Paper Interface specification

```

V {Applicants : Type u_1} {Colleges : Type u_2} [inst : Fintype Applicants] [inst_1 : Fintype Colleges]
[inst_2 : DecidableEq Applicants] [inst_3 : DecidableEq Colleges] (quota : Colleges → ℕ)
(val_applicant : Applicants → Colleges → ℝ) (val_college : Colleges → Applicants → ℝ),
((V (a : Applicants) (c : Colleges), val_applicant a c = val_applicant a c' → c = c') ∧
V (a : Applicants) (c : Colleges), val_applicant a c ≠ 0) →
V
(hcollege_strict :
(V (c : Colleges) (a a' : Applicants), val_college c a = val_college c a' → a = a') ∧
V (c : Colleges) (a : Applicants), val_college c a ≠ 0),
(∃ s,
GS62CollegeAdmissions.ExactCollegeBatchedProcedure.SourceReachable quota val_applicant val_college = s ∧
¬∃ a,
GS62CollegeAdmissions.ExactCollegeBatchedProcedure.SourceActive quota val_applicant val_college = s a) ∧
V (s : GS62CollegeAdmissions.ExactCollegeBatchedProcedure.SourceWaitingListState Applicants Colleges),
GS62CollegeAdmissions.ExactCollegeBatchedProcedure.SourceReachable quota val_applicant val_college = s →
(¬∃ a,
GS62CollegeAdmissions.ExactCollegeBatchedProcedure.SourceActive quota val_applicant val_college = s
a) →
((V (c : Colleges),
((GS62CollegeAdmissions.ExactCollegeBatchedProcedure.sourceAssignmentView quota val_college =
s).college_roster
```

```

      c).card ≤
    quota c) ∧
  (∀ (a : Applicants),
    0 ≤
      EconCSLib.Matching.ManyToOne.valApplicant val_applicant a
      ((GS62CollegeAdmissions.ExactCollegeBatchedProcedure.sourceAssignmentView quota val_college ...
        s).app_match
        a)) ∧
  (∀ (c : Colleges),
    ∀
      a ∈
        (GS62CollegeAdmissions.ExactCollegeBatchedProcedure.sourceAssignmentView quota val_college ...
          s).college_roster
        c,
      0 ≤ val_college c a) ∧
  (∀ (a : Applicants) (c : Colleges),
    EconCSLib.Matching.ManyToOne.valApplicant val_applicant a
      ((GS62CollegeAdmissions.ExactCollegeBatchedProcedure.sourceAssignmentView quota
        val_college ... s).app_match
        a) <
      val_applicant a c →
      (0 < val_college c a ∧
        ((GS62CollegeAdmissions.ExactCollegeBatchedProcedure.sourceAssignmentView quota
          val_college ... s).college_roster
          c).card <
          quota c v
        ∃
          a' ∈
            (GS62CollegeAdmissions.ExactCollegeBatchedProcedure.sourceAssignmentView quota
              val_college ... s).college_roster
            c,
            val_college c a' < val_college c a) →
        False) ∧
    (∀ (c : Colleges), (nu.college_roster c).card ≤ quota c) ∧
    (∀ (a : Applicants),
      0 ≤ EconCSLib.Matching.ManyToOne.valApplicant val_applicant a (nu.app_match a)) ∧
    (∀ (c : Colleges), ∀ a ∈ nu.college_roster c, 0 ≤ val_college c a) ∧
    (∀ (a : Applicants) (c : Colleges),
      EconCSLib.Matching.ManyToOne.valApplicant val_applicant a (nu.app_match a) <
      val_applicant a c →
      (0 < val_college c a ∧ (nu.college_roster c).card < quota c v
        ∃ a' ∈ nu.college_roster c, val_college c a' < val_college c a) →
      False) →
    (∀ (a : Applicants),
      EconCSLib.Matching.ManyToOne.valApplicant val_applicant a (nu.app_match a) ≤
      EconCSLib.Matching.ManyToOne.valApplicant val_applicant a
        ((GS62CollegeAdmissions.ExactCollegeBatchedProcedure.sourceAssignmentView quota val_college ...
          s).app_match
          a)
    )
  )

```

Lean proof endpoint (built separately)

GS62CollegeAdmissions.ProofInterface.theorem2_applicant_optimality

Recorded source-to-Spec screening Verdict: matches

Reason: The raw bundle includes both the earlier displayed replacement-pair condition and Sections 4–5. Read together, the latter uses stable assignment for the terminal top-quota waiting-list outcome and says its stability proof is analogous to the marriage proof; it therefore supplies the completed operational comparison class used by the transparent Spec. The literal display remains a distinct source claim and is not silently identified with that Section 4–5 reading.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation